# Seminar 3. RECURSIVITATE

## I Repere teoretice

### Repere privind executarea funcțiilor recursive

Instrucțiunile aflate înainte de apelurile recursive se execută în ordinea parcurgerii datelor/apelurilor, iar instrucțiunile aflate dupa apelurile recursive se execută la retur, în ordine inversă parcurgerii datelor/apelurilor.

### Repere pentru scrierea unei funcții recursive

1. Funcția trebuie să returneze cel puțin un răspuns direct (pt funcții care returnează valori) sau să aibă o condiție de continuare/oprire pentru funcții void.
2. Orice apel recursiv este condiționat; apelurile trebuie să conducă spre răspunsul direct/condiția de oprire.

## II. Exerciții

1. Pentru programul de mai jos, urmăriți apelurile recursive și scrieți ce se afișează pe ecran, știind că se citesc, în această ordine, numerele 1,2,3,0.

```
void P()
{
      int x;
      scanf("%d",&x);
      if(x) P();
      printf("%d",x);
}
int main()
{
      P();
      return 0;
}
```

| Nr executare | apel | Sursa apel | Instructiuni "tur" | Instrucțiuni "retur1" |
|---|---|---|---|---|
| | main | | P()<br>Paraseste main pt p() | |
| ex1 | P() | main | Citeste x (1)<br>If(1!=0) Da<br>Paraseste ex1 pt P() | Scrie x(1)<br>Stop ex1<br>Revenire la main |
| ex2 | P() | ex1 | Citeste x (2)<br>If(2!=0) Da<br>Paraseste ex2 pt P() | Scrie x(2)<br>Stop ex2<br>Revenire la ex1 |
| ex3 | P() | Ex2 | Citeste x (3)<br>If(3!=0) Da<br>Paraseste ex3 pt P() | Scrie x(3)<br>Stop ex3<br>Revenire la ex2 |
| ex4 | P() | Ex3 | Citeste x (0)<br>If(2!=0) NU<br>Scrie x (0)<br>Stop ex4<br>Revenire in ex3 | |

2. Scrieți de câte ori se execută funcția f în cadrul programului de mai jos.

```
int f(int n)
{
      //printf(„apel\n”);
      if(n<=2) return 1;
      return f(n-1)+f(n-2);
}
int main()
{
      printf(“%d”,f(6));
      return 0;
}
```

| Nr executare | apel | Sursa apel | Instructiuni “tur” | Instrucțiuni “retur1” | Instrucțiuni “retur2” |
|---|---|---|---|---|---|
| | main | | Scrie f(6) Paraseste main pt f(6) | | |
| ex1 | F(n=6) | main | If(6<=2) nu Return f(5)+f(4) Paraseste ex1 pt f(5) | Return 5+f(4) Paraseste exec 1 pt f(4) | |
| ex2 | F(n=5) | ex1 | If(5<=2) nu Return f(4)+f(3) Paraseste exe2 pt f(4) | Return 3+f(3) Paraseste exec 2 pt f(3) | Return 3+2=5 Stop exec 2 cu rezultat 5 Revenire in ex1 |
| ex3 | F(n=4) | Ex2 | If(4<=2) nu Return f(3)+f(2) Paraseste exec3 pt f(3) | Return 2+f(2) Paraseste exc 3 pt f(2) | Return 2+1=3 Stop exec 3 cu rezultat 3 Revenire in ex2 |
| ex4 | F(n=3) | ex3 | If(3<=2) nu Return f(2)+f(1) Paraseste exe4 pt f(2) | Return 1+f(1) Paraseste exec4 pt f(1) | Return 1+1=2 Stop exec4 cu rezultat 2 Revenire in ex3 |
| ex | F(n=2) | Ex4 | If(2<=2)=> return 1 Stop ex5 cu rezultat 1 Revenire in ex4 | | |
| ex6 | F(n=1) | Ex4 | If(1<=2)=>retrun 1 Stop exec6 cu rezultat 1 Revenire in ex4 | | |
| ex7 | F(n=2) | Ex3 | If(2<=2)=>return 1 Stop exec 7 cu rezultat 1 Revenire in ex3 | | |

| | | | | | |
|---|---|---|---|---|---|
| ex8 | F(n=3) | Ex2 | If(3<=2) nu<br>Return f(2)+f(1)<br>Paraseste exec8 pt f(2) | Return 1+f(1)<br>Paraseste exec8 pt f(1) | Return 1+1=2<br>Stop exec 8 cu rezutat 2<br>Revenire in ex2 |
| ex9 | F(n=2) | Ex8 | If(2<=2)=>return 1<br>Stop exec9 cu rezultat 1<br>Revenire in ex8 | | |
| ex10 | F(n=1) | Ex8 | If(1<=2)=>return 1<br>Stop exec 10 cu rezultat 1<br>Revenire in ex8 | | |
| Ex11 | F(n=4) | Ex1 | If(4<=2) nu<br>Return f(3)+f(2)<br>Paraseste exec11 pt f(3) | .... | .... |
| Ex12 | F(n=3) | Ex11 | ..... | | |
| .... | | | | | |
| ..... | | | | | |
| 15 | | | | | |

**III. Probleme rezolvate**

1. Scrieți o funcție recursivă care să calculeze cel mai mare divizor comun a două numere întregi.

Rezolvare:
O variantă de implementare a unei astfel funcții se bazează pe următoarea relație de recurență, obținută din algoritmul lui Euclid:
$cmmdc(a,b)= a$, dacă $b$=0 și *cmmdc(a,b)*= $cmmdc(b,a\%b)$, dacă $b\neq 0$
Implementarea propriu-zisă a funcției este următoarea:

```
int cmmdc(int a, int b)
 {
      if(b == 0) return a;
      return cmmdc(b, a%b);
}
```

Dacă a=b=0, funcția va întoarce valoarea 0. Este corect acest rezultat?

2. Scrieți o funcție recursivă care să calculeze suma valorilor strict pozitive dintr-un tablou unidimensional de numere întregi.

Rezolvare:
O variantă de implementare constă în simularea recursivă a unei instrucțiuni for, folosind parametrul i transmis prin valoare, pentru a parcurge tabloul v de la stânga spre dreapta:

```
int sumasp(int v[], int n, int i)
{
       if(i == n) return 0;
       if(v[i] > 0) return v[i] + sumasp(v, n, i+1);
```

```
        return sumasp(v, n, i+1);
}
```

Funcția recursivă de mai sus se va apela prin `sumasp(v, n, 0)`.

În funcția precedentă parametrul n este necesar pentru a testa dacă au fost parcurse toate elementele tabloului v sau nu. Dacă am parcurge tabloul în sens invers, de la dreapta spre stânga, atunci parcurgerea se termină în momentul în care se ajunge pe poziția 0. Astfel, se poate observa faptul că parametrul n este suficient pentru parcurgerea tabloului, respectiv v[n-1] va reține elementul curent:

```
int sumasp(int v[], int n)
{
      if(n == 0) return 0;
      if(v[n-1] > 0) return v[n-1] + sumasp(v, n-1);
      return sumasp(v, n-1);
}
```

În acest caz funcția recursivă se va apela prin `sumasp(v, n)`.

3. Scrieți o funcție recursivă care să afișeze o matrice t de tip m×n formată din numere întregi.
Rezolvare:
O prima variantă de implementare constă în simularea recursivă a două instrucțiuni for, folosind parametrii i și j, pentru a parcurge matricea pe linii și pe coloane:

```
 void afisare(int t[10][10], int m, int n, int i, int j)
{
        if(i < m)
              if(j < n)
              {
                    printf("%d ", t[i][j]);  afisare(t, m, n, i, j+1);
              }
              else
              {
                    printf("\n"); afisare(t, m, n, i+1, 0);
              }
}
```

Funcția recursivă de mai sus se va apela prin `afisare(t, m, n, 0, 0)`.
Într-un mod asemănător cu cel prezentat în problema anterioară, pentru a elimina cei doi parametrii i și j, trebuie să parcurgem matricea invers, respectiv de la ultima linie spre prima, iar fiecare linie o vom parcurge de la dreapta spre stânga. Mecanismul de stivă asociat recursivității va permite într-un mod implicit inversarea ordinii de afișare a elementelor, deci matricea va fi afișată corect. Cei 2 parametri m și n (reprezentând numărul de linii și numărul de coloane din matrice) îi vom înlocui cu parametrii lcrt și ccrt, reprezentând linia curenta, respectiv coloana curentă. O problemă în acest caz o reprezintă faptul că, după parcurgerea unei linii, trebuie să reluăm parcurgerea liniei aflate deasupra sa de la ultima coloană spre prima, dar nu mai cunoaștem numărul de elemente de pe o linie (valoarea parametrului ccrt fiind modificată). Un artificiu care permite păstrarea numărului de elemente de pe o linie constă în utilizarea unei variabile locale statice ncol care să contorizeze numărul de elemente parcurse pe ultima linie.

```
void afisare(int t[10][10], int lcrt, int ccrt)
{
      static int ncol, pa = 1;
```

```
      if(pa == 1)
       {
             ncol = ccrt + 1;  pa = 0;
       }
      if(lcrt >= 0)
       {
             if(ccrt >= 0)
             {
                    afisare(t, lcrt, ccrt-1);printf("%d ", t[lcrt][ccrt]);
             }
             else
             {
                   afisare(t, lcrt-1, ncol-1); printf("\n");
             }
      }
}
```

Variabila pa (tot statică!) a fost utilizată pentru a restricționa salvarea numărului de coloane în variabila ncol doar în primul apel al funcției.

4. Din numărul 4 se poate obține orice număr natural nenul aplicând în mod repetat următoarele operații:
a) împărțirea numărului la 2;
b) adăugarea cifrei 4 la sfârșitul numărului;
c) adăugarea cifrei 0 la sfârșitul numărului.

De exemplu, numărul 101 se poate obține prin șirul de operații
4→40→404→202→101,

iar numărul 133 prin șirul de operații
4→2→24→12→6→64→32→16→8→ 84→ 42→424→212→106→1064→532→266→133.

Scrieți o funcție recursivă care să afișeze șirul de operații prin care se poate obține un număr natural nenul n din numărul 4.

Rezolvare:
Pentru a afișa șirul de operații prin care se poate obține un număr natural nenul n din numărul 4, vom aplica asupra lui n operațiile inverse operațiilor date:
a') înmulțirea numărului cu 2;
b') eliminarea ultimei cifre, dacă aceasta este 4;
c') eliminarea ultimei cifre, dacă aceasta este 0.

```
void numar4r(int n)
{      if(n != 4)
       {          if(n % 10 == 0 || n%10 == 4) numar4r(n/10);
                  else numar4r(n*2);
                  printf(" -> %d", n);
       }
       else  printf("4");
}
```

O variantă iterativă de implementare a acestui algoritm constă în simularea recursivității folosind o stivă st:

```
void numar4i(int n)
{     int st[1001], k;
      k = 0;     st[0] = n;
       while(st[k] != 4)
       {         if(st[k]%10 == 0 || st[k]%10 == 4) st[++k] = st[k-1]/10;
                  else st[++k] = st[k-1]*2;
        }
        printf("\n");
        while(k > 0)
          printf("%d -> ", st[k--]);
        printf("%d\n", st[0]);
}
```

5. Scrieți o funcție recursivă care să calculeze câte litere mici conține un șir de caractere.

Rezolvare:
Vom testa faptul că litera curentă din șirul s (adică prima literă) este o literă mică folosind funcția islower din biblioteca ctype.h, după care vom reapela funcția pentru restul șirului (care începe cu adresa s+1):

```
int nrLitereMici(char *s)
{     if(*s == '\0')
           return 0;
      if(islower(*s) != 0)
              return 1 + nrLitereMici(s+1);
     return nrLitereMici(s+1);
}
```

Știind că rezultatul evaluării unei expresii de tip logic este 0 sau 1, putem rescrie funcția astfel:

```
 int nrLitereMici(char *s)
{     if(*s == '\0') return 0;
     return (islower(*s) != 0) + nrLitereMici(s+1);
}
```

6. În 1970, John McCarthy, unul dintre întemeietorii inteligenței artificiale, a definit următoarea funcție recursivă, numită funcția McCarthy 91 , pentru a fi utilizată în verificarea formală a corectitudinii algoritmilor: $Mn = n-10$,dacă $n>100$ și $Mn = M(M(n+11))$,dacă $n \leq 100$
Implementați funcția McCarthy 91 atât sub forma unei funcții recursive, cât și sub forma unei funcții iterative.

Rezolvare:
Implementarea recursivă a funcției McCarthy 91 se obține direct din expresia funcției:

```
int mcc91(int n)
{     if(n > 100) return n - 10;
      return mcc91(mcc91(n + 11));
}
```

Implementarea iterativă a funcției McCarthy 91 se poate realiza folosind o stivă pentru a simula mecanismul recursivității:

```
int mcc91(int n)
{     int st[1000], k;
      k = 0;     st[0] = n;
      while(k >= 0)
         if (st[k] <= 100)
         {
             k++;
             st[k] = st[k-1] + 11;
         }
        else
         {
             k--;
             if (k>=0) st[k] = st[k+1] - 10;
         }
     return st[0] - 10;
 }
```

O implementare foarte simplă și eficientă a funcției McCarthy 91 se obține dacă se observă (și se demonstrează matematic!) faptul că ea este echivalentă cu următoarea funcției: $Mn$=$n$−10,dacă $n$>100 91,dacă $n$≤100

**IV. Probleme propuse**

1. Scrieți câte o funcție recursivă care să afișeze divizorii naturali proprii ai unui număr natural n
a) în ordine crescătoare.
b) în ordine descrescătoare (în trei variante: pe măsura parcurgerii datelor, la retur, tur-retur).

2. Scrieți o funcție recursivă care să returneze numărul de divizori naturali proprii ai unui număr natural n.

3. Scrieți o funcție recursivă care să returneze suma divizorilor naturali proprii ai unui număr natural n

4. Scrieți o funcție recursivă care să calculeze valoarea maximă dintr-un tablou unidimensional de numere întregi.

5. Scrieți o funcție recursivă care să calculeze de câte ori apare o literă dată într-un șir de caractere.

6. Scrieți o funcție recursivă care să verifice dacă două șiruri de caractere sunt anagrame sau nu.

7. Scrieți o funcție recursivă care să verifice dacă un număr natural este palindrom sau nu.

8. Algoritmi FILL
O suprafață de formă dreptunghiulară este memorată ca o matrice. Pe suprafață este figurată o curbă închisă. Dându-se un punct în interiorul suprafeței mărginite de curbă, se cere hașurarea acesteia.
Date de intrare: fișierul fill.in conține pe prima linie patru numere naturale, nrl, nrc, i0,j0
1≤i0≤nrl≤100, 1≤j0≤nrc≤100, iar pe următoarele nrl linii, elementele matricei.
Elementele matricei care aparțin curbei sunt notate cu 1, iar celelalte sunt notate cu 0.
Punctul din interiorul suprafeței care trebuie hașurată este pe linia i0 și coloana j0.
Date de ieșire: în fișierul fill.out se vor afla elementele matricei după hașurarea suprafeței cerute.

9. O fotografie este memorată ca o matrice binară. Elementele care aparțin obiectelor din fotografie sunt memorate ca 1, iar cele care aparțin fundalului, sunt notate cu 0. Știind că obiectele nu se ating, determinați numărul acestora.

10. Relațiile lui Viete
Se citesc trei numere reale, a,b și c, și un număr natural n. Scrieți o funcție recursivă care să calculeze valoarea expresiei
$x_1^n+x_2^n$
Știind că $x_1$ și $x_2$ sunt soluțiile ecuației
$ax^2+bx+c=0$

$ax1^2+bx1+c=0 \mid x1^{n-2}$
$ax2^2+bx2+c=0 \mid x2^{n-2}$

$a(x1^n+x2^{n-1})+b(x1^{n-1}+x2^{n-1})+c(x1^{n-2}+x2^{n-2})=0$
$a* S(n)+b*S(n-1)+c*S(n-2)=0$
$S(n)=-b/a * S(n-1) -c/a * S(n-2)$

$S(1)=x1+x2$
$S(0)=1+1=2$

```
float s(int n)
{
  if(n==0) return 2;
  if(n==1) return x1+x2;
  return -b/a*s(n-1)-c/a*s(n-2);
}
```